



LeetCode 743 — Network Delay Time

1. Problem Title & Link

Title: LeetCode 743: Network Delay Time

Link: <https://leetcode.com/problems/network-delay-time/>

2. Problem Statement (Short Summary)

You are given a list of directed weighted edges: $\text{times}[i] = (u, v, w)$

Meaning:

👉 A signal takes **w** time to travel from node **u** to node **v**.

You have:

- n total nodes labeled 1...n
- A starting node k

Your job:

👉 Return **minimum time** for the signal from k to reach **all nodes**.

If *even one* node cannot be reached → return -1.

3. Examples (Input → Output)

Example 1

Input:

$\text{times} = [[2,1,1],[2,3,1],[3,4,1]]$

$n = 4$

$k = 2$

Output:

2

Explanation:

$2 \rightarrow 1 = 1$

$2 \rightarrow 3 = 1$

$2 \rightarrow 3 \rightarrow 4 = 1 + 1 = 2$

Maximum delay = 2

Example 2

Input:

$\text{times} = [[1,2,1]]$

$n = 2$

$k = 1$

Output: 1

**Example 3**

Input:

times = [[1,2,1]]

n = 2

k = 2

Output: -1

Node 1 is unreachable from 2.

4. Constraints

- $1 \leq n \leq 100$
- edges ≤ 6000
- weights > 0
- Graph is directed
- Must find **shortest path to all nodes**
- Classic Dijkstra problem

5. Core Concept (Pattern / Topic)

Graph + Dijkstra's Algorithm (Min-Heap Priority Queue)

Single-source shortest path in a weighted directed graph.

6. Thought Process (Step-by-Step)**✗ Brute Force**

Try all paths → exponential and impossible.

✓ Optimized — Dijkstra (Best)

Why Dijkstra?

- All edge weights are **positive**
- We need **shortest time** from a starting node to all others
- Answer = the **maximum** shortest time (the slowest node to receive signal)

Steps:

1. Build adjacency list
2. Use min-heap
3. Dist array initialized with infinity
4. Dijkstra to compute min-distance to every node
5. If any node remains unreachable → return -1



6. Else return max(distances)

7. Visual / Intuition Diagram

Graph:

$2 \rightarrow 1$ (1)

$2 \rightarrow 3$ (1)

$3 \rightarrow 4$ (1)

Shortest paths:

$\text{dist}[1] = 1$

$\text{dist}[3] = 1$

$\text{dist}[4] = \text{dist}[3] + 1 = 2$

Max delay $\rightarrow 2$.

8. Pseudocode

```
build graph adjacency list

dist[] = infinity
dist[k] = 0

minHeap = [(0, k)]

while heap not empty:
    time, node = pop minimum

    for (nextNode, weight) in graph[node]:
        if time + weight < dist[nextNode]:
            dist[nextNode] = time + weight
            push (dist[nextNode], nextNode) into heap

if any dist[i] == infinity:
    return -1

return max(dist)
```

9. Code Implementation

✓ Python

```
import heapq
from collections import defaultdict
```



```
class Solution:
    def networkDelayTime(self, times, n, k):
        graph = defaultdict(list)
        for u, v, w in times:
            graph[u].append((v, w))

        dist = [float('inf')] * (n + 1)
        dist[k] = 0

        min_heap = [(0, k)]  # (time, node)

        while min_heap:
            time, node = heapq.heappop(min_heap)

            if time > dist[node]:
                continue

            for nei, w in graph[node]:
                new_time = time + w
                if new_time < dist[nei]:
                    dist[nei] = new_time
                    heapq.heappush(min_heap, (new_time, nei))

        max_time = max(dist[1:])
        return -1 if max_time == float('inf') else max_time
```

✓ Java

```
import java.util.*;

class Solution {
    public int networkDelayTime(int[][] times, int n, int k) {
        List<List<int[]>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) graph.add(new ArrayList<>());

        for (int[] t : times) {
            graph.get(t[0]).add(new int[]{t[1], t[2]});
        }

        int[] dist = new int[n + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;
```



```

PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
pq.offer(new int[]{0, k}); // {time, node}

while (!pq.isEmpty()) {
    int[] curr = pq.poll();
    int time = curr[0], node = curr[1];

    if (time > dist[node]) continue;

    for (int[] nei : graph.get(node)) {
        int next = nei[0], w = nei[1];
        int newTime = time + w;

        if (newTime < dist[next]) {
            dist[next] = newTime;
            pq.offer(new int[]{newTime, next});
        }
    }
}

int max = 0;
for (int i = 1; i <= n; i++) {
    if (dist[i] == Integer.MAX_VALUE) return -1;
    max = Math.max(max, dist[i]);
}

return max;
}
}

```

10. Time & Space Complexity

Time

- Dijkstra using heap: **$O(E \log V)$**
(At worst $6000 \log 100$)

Space

- Graph: **$O(E)$**
- Heap + dist: **$O(V)$**

11. Common Mistakes / Edge Cases

✗ Mistakes

- Using BFS instead of Dijkstra (because weights $\neq 1$)



- Forgetting to check for unreachable nodes
- Not using min-heap → TLE
- Incorrect indexing for 1-based nodes

✓ Edge Cases

- Graph fully disconnected
- Some nodes unreachable from k
- $n = 1$
- multiple edges between same nodes

12. Detailed Dry Run

Input:

times = $[[2,1,1],[2,3,1],[3,4,1]]$

$n=4, k=2$

Pop	Update	Distances
(0,2)	set $\text{dist}[1]=1, \text{dist}[3]=1$	$[\infty, 1, 0, 1, \infty]$
(1,1)	no outgoing edges	—
(1,3)	$\text{dist}[4]=2$	$[\infty, 1, 0, 1, 2]$
(2,4)	no edges	—

Max dist = **2**

13. Common Use Cases (Real-Life / Interview)

- Network packet delay simulation
- Finding minimum time for broadcast
- Routing problems
- Shortest communication time

14. Common Traps

- Updating heap but not updating $\text{dist}[]$
- Storing edges incorrectly
- Using adjacency matrix (slow) instead of list
- Not checking for unreachable nodes

15. Builds To (Related LeetCode Problems)

- LC 787 — Cheapest Flights Within K Stops
- LC 1514 — Path with Maximum Probability
- LC 1631 — Path With Minimum Effort



- LC 1334 — Find City With Fewest Neighbors
- LC 2136 — Maximum Running Time of N Computers

16. Alternate Approaches + Comparison

1 Bellman-Ford

- Handles negative weights
- But slower $\rightarrow O(VE)$
- Not needed here since weights are positive.

2 Floyd-Warshall

- All-pairs shortest path
- $O(n^3) \rightarrow$ impossible for $n=100$ here

✓ 3 Dijkstra (BEST)

- Handles positive weights
- Fastest and optimal

17. Why This Solution Works (Short Intuition)

Dijkstra always processes the currently fastest path.

Since weights are positive, once a node's shortest distance is fixed, it never changes.

The largest of these shortest distances is the answer.

18. Variations / Follow-Up Questions

- What if edges had negative weights? (Use Bellman-Ford)
- What if you need the actual shortest path tree?
- How to detect unreachable nodes early?
- Can we do it using BFS? (Only if weight = 1 everywhere)